

Implementasi Awal CRUD: Campus Event Manager

Topik: Campus Event Management System

Kelompok: 11

Github: github.com/Dzaky-Ar/DBProject_Group11_CampusEventManager

```
root/
├── Configuration/           # Berisi konfigurasi server dan database
│   └── config.php          # File koneksi database
├── Main_menu/              # Modul autentikasi dan menu utama
│   ├── Login_Page.php      # Halaman login pengguna
│   ├── Register_Page.php   # Halaman registrasi pengguna
│   └── Mapping.php         # Logika autentikasi dan proses CRUD
└── Dashbaord/              # Halaman dashboard setelah login
```

1. Operasi Create: Registrasi Pengguna dan Pengajuan Event

Operasi *Create* menjadi tahap awal interaksi pengguna dengan sistem. Pada proses registrasi, pengguna memasukkan data dasar seperti nama, email, password, dan status peran. Data ini kemudian diproses dan disimpan ke dalam tabel *User*. Jika pengguna mendaftar sebagai organizer, sistem secara otomatis menambahkan data terkait ke tabel *Organizer*, sehingga relasi antar tabel dapat terjaga sejak awal.

Selain registrasi pengguna, operasi *Create* juga diterapkan pada dashboard pengguna ketika melakukan pengajuan event. Data event, lokasi, kebutuhan inventaris, serta dokumen pendukung seperti proposal dalam bentuk file PDF disimpan ke dalam tabel *Registration*. Integrasi ini memungkinkan satu proses submit menghasilkan beberapa entri data yang saling berelasi di dalam basis data.

2. Operasi Read: Autentikasi dan Visualisasi Data

Operasi *Read* berperan penting dalam proses login dan penampilan data. Pada saat login, sistem mengambil data pengguna berdasarkan email untuk keperluan autentikasi dan penentuan hak akses. Setelah berhasil masuk, pengguna diarahkan ke dashboard sesuai perannya.

Pada dashboard verifactor, operasi *Read* menjadi lebih kompleks karena melibatkan pengambilan data dari beberapa tabel yang saling terhubung, seperti *Organizer*, *Event*, *Location*, *Inventory*, dan *borrow*s. Integrasi ini memungkinkan verifactor melihat informasi proposal secara menyeluruh, termasuk detail event, kebutuhan barang, jumlah, serta periode peminjaman. Dengan demikian, sistem mampu menyajikan data yang komprehensif dan konsisten dalam satu tampilan.

3. Operasi Update: Perubahan Status dan Data Submission

Operasi *Update* digunakan untuk memodifikasi data yang sudah ada. Pada sisi pengguna, operasi ini memungkinkan pembaruan data pengajuan event, misalnya ketika revisi proposal diperlukan. Perubahan tersebut langsung tercermin pada tabel *Registration* dan tabel terkait lainnya.

Pada dashboard verifactor, operasi *Update* digunakan untuk mengubah status pengajuan event, seperti dari *pending* menjadi *approved* atau *rejected*. Perubahan status ini menjadi penghubung utama antara peran verifactor dan pengguna, karena menentukan kelanjutan proses event dalam sistem.

4. Operasi Delete: Pembatalan dan Manajemen Data

Operasi *Delete* diterapkan untuk mendukung fleksibilitas pengelolaan data. Pengguna dapat membatalkan pengajuan event yang telah dibuat, sehingga data terkait dihapus dari tabel *Registration*. Operasi ini membantu menjaga konsistensi data serta mencegah penumpukan data yang sudah tidak relevan.

5. Integrasi CRUD dalam Arsitektur Sistem

Integrasi operasi CRUD dalam sistem ini menunjukkan bagaimana setiap fitur saling terhubung secara logis dan fungsional. File seperti *config.php* berperan sebagai pusat koneksi basis data, sementara *Mapping.php* dan berbagai file dashboard mengatur alur data sesuai peran pengguna. Struktur basis data yang dirancang dalam *CEM_schema.sql* mendukung relasi antar entitas secara efisien, sehingga operasi CRUD dapat berjalan tanpa redundansi data.